Real-Time Face Filter and Deepfake Detection System

Lachi Mohamed Amine

2026

Contents

1 Introduction **2**

2 Problem Definition **2**

3 System Architecture **2**

4 Technology Stack **3**
- 4.1 Programming Language . . . 3
- 4.2 Deep Learning Frameworks . . . 3
- 4.3 Computer Vision Libraries . . . 3

5 Dataset Collection **3**
- 5.1 FaceForensics++ . . . 4
- 5.2 Celeb-DF . . . 4
- 5.3 DeepFake Detection Challenge Dataset . . . 4
- 5.4 Synthetic Filter Dataset . . . 4

6 Face Extraction **4**

7 Data Preprocessing **4**

8 Model Architecture **5**

9 Training Procedure **5**

10 Temporal Analysis **5**

11 Real-Time Detection **6**

12 Deployment **6**

13 Evaluation Metrics **6**

14 Hardware Requirements **7**

15 Project Timeline **7**

16 Conclusion **7**

1 Introduction

With the rapid evolution of artificial intelligence and generative models, face manipulation technologies such as deepfakes and real-time beauty filters have become widely accessible. These technologies are commonly used in social media platforms and video communication systems.

Applications such as WhatsApp, Snapchat, Instagram, and TikTok allow users to modify facial appearance using augmented reality filters or deepfake-based techniques. While these technologies provide entertainment value, they also introduce concerns related to misinformation, identity manipulation, and digital trust.

This project proposes the development of an open-source artificial intelligence system capable of detecting facial manipulation in real-time video streams. The system focuses on identifying:

- Real faces
- Faces modified with beauty or AR filters
- AI-generated deepfake faces

The objective is to design a pipeline capable of analyzing faces in video-call scenarios and producing a confidence score indicating whether the face is authentic or manipulated.

2 Problem Definition

The primary goal of this project is to design a deep learning model capable of detecting facial manipulation in real-time environments.

The system must:

- Extract faces from live video streams
- Detect visual artifacts introduced by filters or deepfake generation
- Classify frames into three categories:
  - Real
  - Filtered
  - Deepfake
- Provide real-time predictions suitable for video calls

3 System Architecture

The proposed system follows a multi-stage pipeline:

1. Video acquisition
2. Face detection
3. Face extraction
4. Feature extraction

5. Deep learning classification
6. Temporal analysis
7. Real-time output

The processing pipeline can be summarized as follows:

```
Video Stream
     ↓
Face Detection
     ↓
Face Extraction
     ↓
Feature Extraction
     ↓
Deepfake / Filter Classifier
     ↓
Probability Score
```

4 Technology Stack

The project relies entirely on open-source tools.

4.1 Programming Language

Python is used due to its extensive machine learning ecosystem.

4.2 Deep Learning Frameworks

Two major frameworks can be used:

- PyTorch
- TensorFlow

PyTorch is recommended due to its flexibility and strong research community.

4.3 Computer Vision Libraries

The following tools are used for face processing:

- OpenCV for image processing
- MediaPipe for face detection and facial landmarks

5 Dataset Collection

Training an accurate model requires large datasets containing both authentic and manipulated faces.

5.1 FaceForensics++

FaceForensics++ is one of the most widely used datasets for deepfake research. It contains both real videos and manipulated videos produced using various deepfake generation techniques.

The dataset includes over 1.8 million frames extracted from videos.

5.2 Celeb-DF

Celeb-DF provides high-quality deepfake videos generated using improved techniques designed to produce more realistic manipulations.

It is particularly useful for evaluating the robustness of detection models.

5.3 DeepFake Detection Challenge Dataset

This dataset was released for the DeepFake Detection Challenge and contains over one hundred thousand videos.

It includes a large variety of real and manipulated videos suitable for training large-scale models.

5.4 Synthetic Filter Dataset

To detect beauty filters, a custom dataset must be created.

This dataset can be generated by applying filters from social media platforms to real images and recording both the original and filtered versions.

The labeling format can be defined as follows:

```
0 = real
1 = filter
2 = deepfake
```

6 Face Extraction

Videos are first converted into individual frames. Face detection algorithms are then used to locate faces within each frame.

Once detected, the facial region is cropped and saved for training.

The dataset structure should follow this format:

```
dataset/
   real/
   deepfake/
   filter/
```

7 Data Preprocessing

All faces are resized to a consistent resolution.

A common input size for convolutional neural networks is:

```
224 × 224 pixels
```

Normalization is applied using standard ImageNet statistics.

Data augmentation techniques are used to increase model robustness:

- horizontal flipping
- brightness variation
- compression artifacts
- gaussian blur

8 Model Architecture

Several deep neural network architectures have proven effective for deepfake detection.

Examples include:

- ResNet
- EfficientNet
- Xception

EfficientNet is particularly suitable due to its balance between performance and computational efficiency.

A simple classification model outputs three probabilities corresponding to the three classes.

9 Training Procedure

The training configuration may include:

- Loss Function: CrossEntropyLoss
- Optimizer: AdamW
- Batch Size: 32
- Training Epochs: 20–40

During training, the model learns to distinguish real facial textures from artifacts introduced by filters or generative models.

10 Temporal Analysis

Deepfake artifacts are often more visible across multiple frames rather than a single frame.

To capture temporal inconsistencies, sequential models can be used:

- Long Short-Term Memory (LSTM)
- 3D Convolutional Neural Networks

These models analyze frame sequences and detect unnatural temporal patterns.

11 Real-Time Detection

For deployment in video-call scenarios, the system must process frames in real time.

The inference pipeline is:

```
Camera Input
     ↓
Face Detection
     ↓
Face Cropping
     ↓
Neural Network Inference
     ↓
Manipulation Score
```

Example output:

```
REAL: 0.85
FILTER: 0.10
DEEPFAKE: 0.05
```

12 Deployment

For mobile or edge deployment, the trained model can be converted into lightweight formats.

Common options include:

- TensorFlow Lite
- ONNX Runtime

These formats enable execution on mobile devices and embedded systems.

13 Evaluation Metrics

The performance of the model can be evaluated using several metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- Area Under the ROC Curve (AUC)

Testing should be performed on datasets not used during training to ensure proper generalization.

14 Hardware Requirements

Training deep learning models requires GPU acceleration.

Recommended configuration:

- GPU with at least 8GB VRAM
- 16GB system memory
- CUDA-compatible environment

Cloud-based environments such as Google Colab and Kaggle can also be used for training.

15 Project Timeline

A possible development schedule is:

- Week 1: Dataset collection and face extraction
- Week 2: Data preprocessing and augmentation
- Week 3: CNN model training
- Week 4: Real-time inference implementation
- Week 5: Temporal detection integration

16 Conclusion

This project presents an open-source roadmap for building a real-time system capable of detecting face filters and deepfake manipulations in video streams.

By combining modern deep learning architectures with computer vision techniques, it is possible to build systems capable of detecting facial manipulation with high accuracy.

Such systems can contribute to improving digital trust and preventing the misuse of generative media technologies in communication platforms.